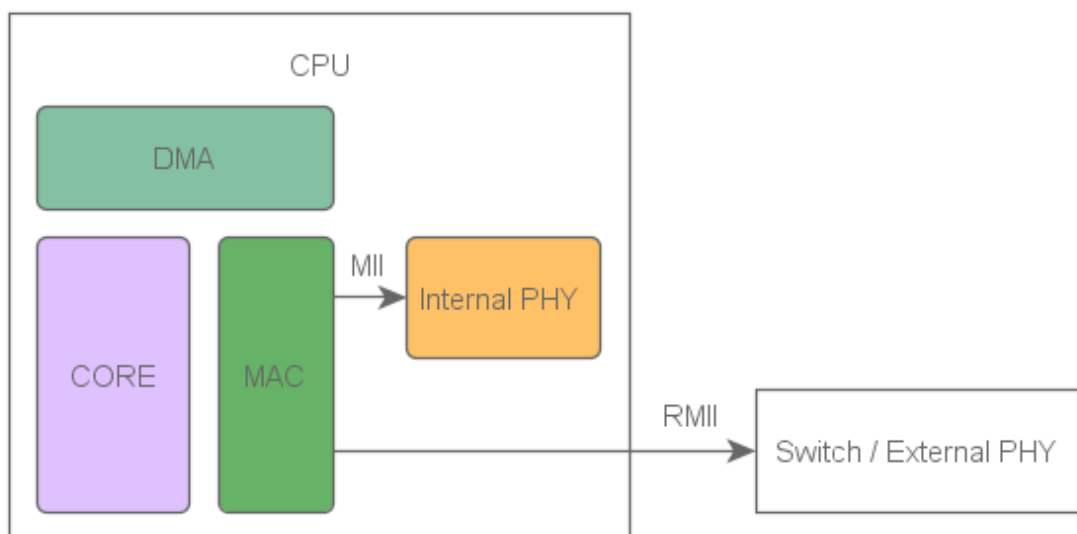


SSD_Ethernet usage reference

1 Overview

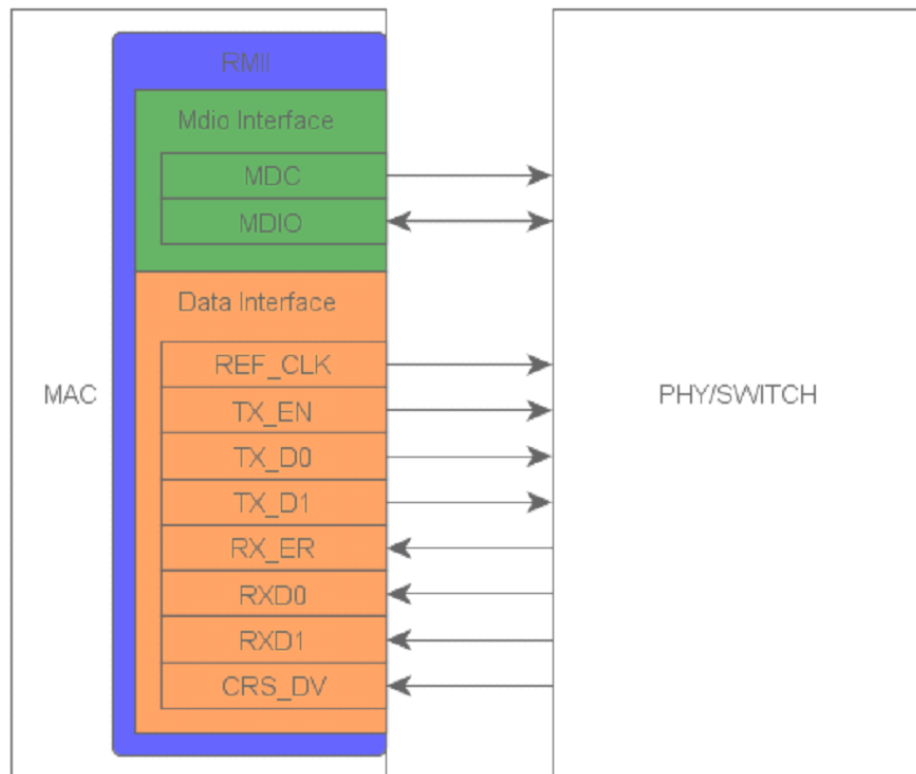
1.1. Introduction to Ethernet Architecture

From the hardware point of view, the Ethernet interface circuit is mainly composed of a MAC (Media Access Control) controller and a physical layer interface PHY (Physical Layer, PHY). The Sigmastar platform has a built-in MAC controller and built-in PHY; it also supports the RMI bus protocol, which can be connected to an external switch or PHY chip. The diagram is as follows:



1.2. RMII interface introduction

The RMII interface includes a data interface and a management interface between the MAC and the PHY. The MDIO management interface includes two signal lines (MDC and MDIO). Through this interface, the MAC can read the PHY or Switch chip registers to achieve transmission information and status control.

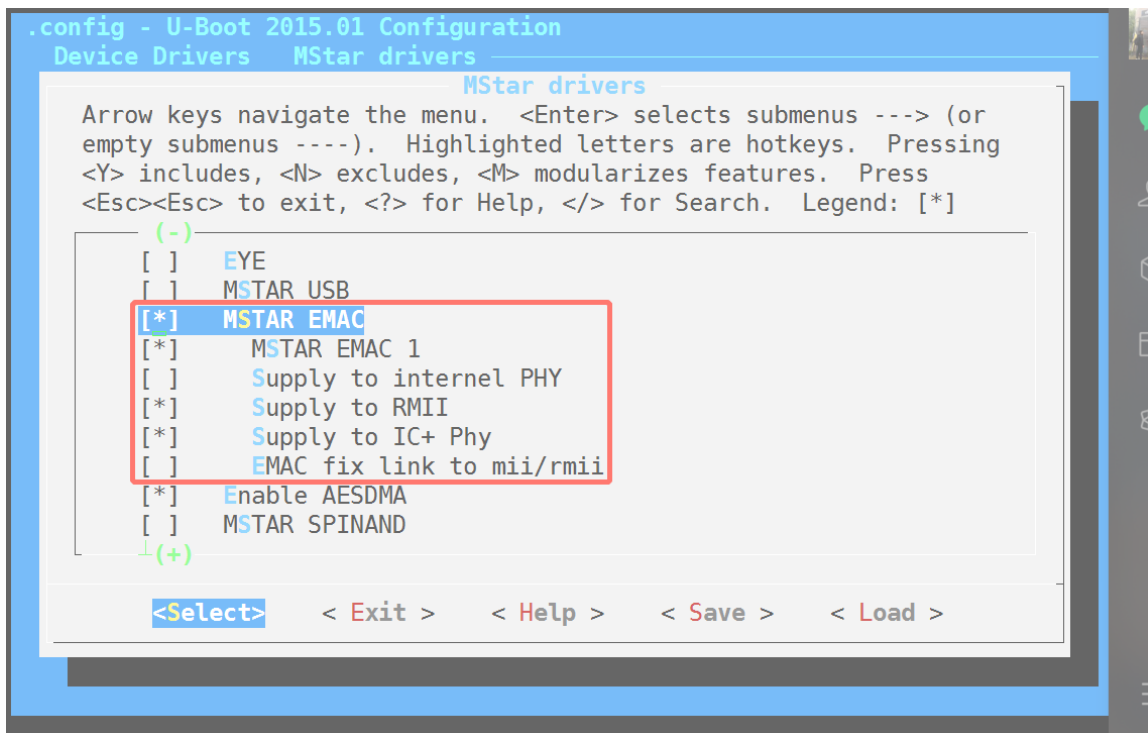


1.3. Using RMII under uboot

SSD2XX Platform Example

1.3.1 Enable RMII support under uboot

Enable emac in menuconfig, close the built-in phy and open the rmii interface, as follows:



1.3.2 Use MDIO to initialize PHY or Switch

The mdio read and write operation functions are provided under uboot, see the file: boot/drivers/mstar/emacs/pioneer3/mhal_emac.c

MDIO read:

```
void MHal_EMAC_read_phy(unsigned char phy_addr, unsigned char
address, u32 *value)
```

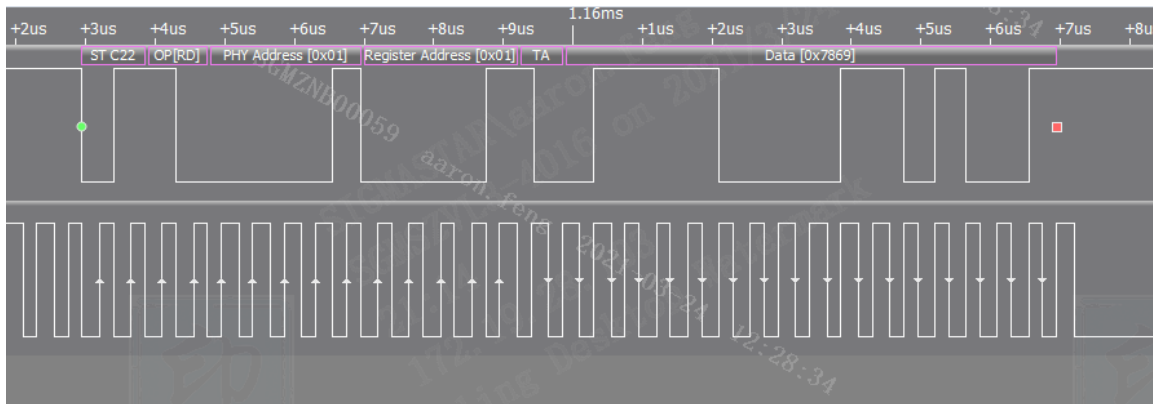
MDIO writes:

```
void MHal_EMAC_write_phy (unsigned char phy_addr, unsigned char
address, u32 value)
```

Refer to the ip101a_g_config_init function in the boot/drivers/mstar/emacs/mdrv_emac.c file to initialize the external PHY or Switch chip using mdio read/write access registers.

It should be noted that during the read operation, the MDC clock on the master side will become the falling edge sampling after the TA signal, as shown in the

figure below; the write operation will always keep the rising edge sampling. To determine whether the MDIO is read and written correctly, a logic analyzer can be used to capture the MDIO and MDC signal waveforms.



1.4. Using RMII under the kernel

SSD20X platform example

1.4.1 Enable network support in kernel

Open the following options in menuconfig:

```
config - Linux/arm 4.9.84 Kernel Configuration
> Device Drivers > Network device support > PHY Device support and infrastructure
  Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus ---). Hig
  Pressing <Y> includes, <N> excludes, <M> modularizes features. Press <Esc><Esc> to exit, <
  Legend: [*] built-in [ ] excluded <M> module < > module capable

  ^(-)
  < > Texas Instruments DP83867 Gigabit PHY
  -* MDIO Bus/PHY emulation with fixed speed/link PHYs
  <*> ICPlus PHYs
  < > Intel XWAY PHYs
  < > LSI ET1011C PHY
  < > Intel LXT PHYs
  < > Marvell PHYs
```

```
config - Linux/arm 4.9.84 Kernel Configuration
> Device Drivers > SStar SoC platform drivers
  Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus ---
  Pressing <Y> includes, <N> excludes, <M> modularizes features. Press <Esc><Esc> to
  Legend: [*] built-in [ ] excluded <M> module < > module capable

  ^(-)
  <M> SStar SD/MMC Card Interface Support
  <*> EMAC
  [ ] Supply to random number generator device
  [ ] Supply to hardware TX checksum
  [ ] Supply to K3 RX SW Patch
  (1) Disconnect delay in second
  [ ] Supply to EEE function
```

1.4.2 Configure RMII pin PINMUX

Open the kernel\arch\arm\boot\dts\infinity2m-xxxxx-padmux-display.dtsi file, and add the following content to the padmux field:

```
<PAD_TTL16      PINMUX_FOR_ETH1_MODE_3      MDRV_PUSE_NA >,
<PAD_TTL17      PINMUX_FOR_ETH1_MODE_3      MDRV_PUSE_NA >,
<PAD_TTL18      PINMUX_FOR_ETH1_MODE_3      MDRV_PUSE_NA >,
<PAD_TTL19      PINMUX_FOR_ETH1_MODE_3      MDRV_PUSE_NA >,
<PAD_TTL20      PINMUX_FOR_ETH1_MODE_3      MDRV_PUSE_NA >,
<PAD_TTL21      PINMUX_FOR_ETH1_MODE_3      MDRV_PUSE_NA >,
<PAD_TTL22      PINMUX_FOR_ETH1_MODE_3      MDRV_PUSE_NA >,
<PAD_TTL23      PINMUX_FOR_ETH1_MODE_3      MDRV_PUSE_NA >,
<PAD_TTL24      PINMUX_FOR_ETH1_MODE_3      MDRV_PUSE_NA >,
<PAD_TTL25      PINMUX_FOR_GPIO_MODE
MDRV_PUSE_EMAC1_PHY_RESET>,
```

1.4.3 修改设备树使能RMII

打开kernel\arch\arm\boot\dts\infinity2m.dtsi文件，修改如下内容，配置phy-mode为rmii接口。

```
emac1: emac1 {
    compatible = "sstar-emac";
    interrupts = <GIC_SPI INT_IRQ_EMAC_1
IRQ_TYPE_LEVEL_HIGH>, <GIC_SPI INT_FIQ_LAN_ESD
IRQ_TYPE_LEVEL_HIGH>;
    clocks = <&CLK_emac_ahb>, <&CLK_emac1_tx>,
<&CLK_emac1_rx>, <&CLK_emac1_tx_ref>, <&CLK_emac1_rx_ref>;
    reg = <0x1F2A2800 0x800>, <0x1F344200 0x600>,
<0x00000000 0x000>;
    pad = <0x1F203C38 0x0F00 0x0300>; // pad selection from
0x0100/0x0200/0x0300/0x0400/0x0500/0x0600/0x0700/0x0800/0x0900
    status = "ok";
    #if 1
        phy-handle = <&phy1>;
        mdio-bus@emac1 {
            phy1: ethernet-phy@1 {
                phy-mode = "rmii";
            };
        };
    #else
```

```
        phy-mode = "rmii";
        fixed-link = <0 1 100 0 0>;
    #endif
```

1.4.4 通过MDIO接口初始化PHY或者Switch

Kernel下同样支持mdio读写操作，如下：

MDIO读函数：

```
MDev_EMAC_mii_read
```

MDIO写函数：

```
MDev_EMAC_mii_write
```

MII/RMII初始化函数：

```
MDev_EMAC_mii_init
```

具体代码见kernel/drivers/ssstar/emac/mdrv_emac.c文件。用户初始化PHY或者Switch芯片的函数可添加在MDev_EMAC_mii_init函数之后。

```
static int MDev_EMAC_mii_write(struct mii_bus *bus, int phy_addr,
int phy_reg, u16 val)
{
    struct emac_handle *hemac = (struct emac_handle *) bus->priv;
    int ret;

    ret = MHal_EMAC_write_phy(hemac->hal, phy_addr, phy_reg,
(u32)val);
    return ret;
}

static int MDev_EMAC_mii_read(struct mii_bus *bus, int phy_addr,
int phy_reg)
{
    u32 val;
    struct emac_handle *hemac = (struct emac_handle *) bus->priv;
    int ret;

    ret = MHal_EMAC_read_phy(hemac->hal, phy_addr, phy_reg, &val);
```

```

        return (int)val;
    }

    static int MDev_EMAC_mii_init(struct net_device* emac_dev)
    {
        struct emac_handle *hemac = (struct emac_handle *)
netdev_priv(emac_dev);
        struct device_node *mii_np = NULL;
        int ret = 0;

        ...

        hemac->mii_bus->name = "mdio";
        hemac->mii_bus->read = MDev_EMAC_mii_read;
        hemac->mii_bus->write = MDev_EMAC_mii_write;
        hemac->mii_bus->priv = hemac;
        hemac->mii_bus->parent = hemac->dev;

        ...

        return ret;
    }

```

2. ETHERNET常用配置说明

2.1. uboot与kernel下配置IP地址

uboot下使用如下命令设置板端IP与电脑端IP，并使用ping命令测试网络是否连通：

```

SigmaStar # setenv ipaddr 192.168.50.123

SigmaStar # setenv serverip 192.168.50.249

SigmaStar # ping 192.168.50.249

Using sstar_emac device

host 192.168.50.249 is alive

```

kernel下使用ifconfig命令配置IP相关属性：

```
/ # ifconfig eth0 up

/ # ifconfig eth0 hw ether 00:70:27:00:00:03

/ # ifconfig eth0 192.168.50.123 netmask 255.255.255.0

/ # route add default gw 192.168.50.1
```

2.2. 使用DHCP自动获取IP

脚本文件位于工程目录project/image/etc/init.d/udhcpd.script，打包image时会拷贝到板端，kernel起来后板端运行udhcpd命令，可自动获取ip，示例如下：

```
/ # udhcpd -i eth0 -s /etc/init.d/udhcpd.script
udhcpd (v1.20.2) started
Setting IP address 0.0.0.0 on eth0
Sending discover...
Sending discover...
Sending select for 192.168.50.243...
Lease of 192.168.50.243 obtained, lease time 86400
Setting IP address 192.168.50.243 on eth0
Deleting routers
route: SIOCDELRT: No such process
Adding router 192.168.50.1
Recreating /appconfigs/resolv.conf
Adding DNS server 192.168.50.1
```

2.3. DNS服务使用介绍

2.3.1 准备DNS依赖库

打开DNS服务需要依赖相关的库文件，可以从toolchain中拷贝到板端，使用export命令导出动态库。

```
/ # export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/lib/libdns
```

示例，如果使用的是9.1.0的toolchain，则相关的库文件在如下目录：


```
/tools/toolchain/gcc-sigmastar-9.1.0-2020.07-x86_64_arm-linux-  
gnueabihf/arm-linux-gnueabihf/libc/lib/
```

```
libnsl-2.30.so  
libnsl.so.1  
libnss_compat-2.30.so  
libnss_compat.so.2  
libnss_db-2.30.so  
libnss_db.so.2  
libnss_dns-2.30.so  
libnss_dns.so.2  
libnss_files-2.30.so  
libnss_files.so.2  
libnss_hesiod-2.30.so  
libnss_hesiod.so.2  
libresolv-2.30.so  
libresolv.so.2
```

2.3.2 配置DNS服务器

在板端创建/etc/resolv.conf配置文件，并写入对应的dns server ip，如：

```
/ # echo "nameserver 8.8.8.8" >> /etc/resolv.conf
```

2.3.3 测试DNS

使用ping命令测试dns解析域名，如：

```
/ # ping www.baidu.com  
PING www.baidu.com (163.177.151.109): 56 data bytes  
64 bytes from 163.177.151.109: seq=0 ttl=55 time=7.640 ms  
64 bytes from 163.177.151.109: seq=1 ttl=55 time=8.002 ms  
64 bytes from 163.177.151.109: seq=2 ttl=55 time=7.443 ms  
--- www.baidu.com ping statistics ---  
3 packets transmitted, 3 packets received, 0% packet loss  
round-trip min/avg/max = 7.443/7.695/8.002 ms
```

3. ETHERNET带宽测试说明

3.1. 使用iperf测试网络性能

3.1.1 下载iperf源码

linux端下载地址：

<https://iperf.fr/download/source/iperf-2.0.9-source.tar.gz>

[<https://iperf.fr/download/source/iperf-2.0.9-source.tar.gz>]

PC端下载地址：

<https://iperf.fr/iperf-download.php#windows> [<https://iperf.fr/iperf-download.php#windows>]

注意：linux端与PC端的iperf版本保持一致

3.1.2 编译iperf

配置交叉编译器：

```
./configure --help  
./configure --prefix=$(pwd)/host --host=arm-linux-gnueabihf  
CXX=arm-linux-gnueabihf-g++ CC=arm-linux-gnueabihf-gcc
```

编译并安装，完成后会在当前目录host/bin下生成可执行档iperf，拷贝到板端：

```
make clean  
make -j4  
make install
```

3.1.3 测试网络

帮助命令：

```
/ # ./iperf -h
```

-f [k|m|K|M] 分别表示以Kbits, Mbits, KBytes, MBytes显示报告, 默认以Mbits为单位

-i sec 以秒为单位显示报告间隔, eg: iperf -c 222.35.11.23 -i 2

-l 缓冲区大小, 默认是8KB, eg: iperf -c 222.35.11.23 -l 16 -m 显示tcp最大mtu值

-o 将报告和错误信息输出到文件, eg: iperf -c 222.35.11.23 -o /mnt/iperflog.txt

-p 指定服务器端使用的端口或客户端所连接的端口

-u 使用udp协议

-w 指定TCP窗口大小, 默认是8KB

-M 设定TCP数据包的最大mtu值

client端专用参数 :

-d 同时进行双向传输测试

-n 指定传输的字节数, eg: iperf -c 222.35.11.23 -n 100000

-r 单独进行双向传输测试

-t 测试时间,默认10秒, eg: iperf -c 222.35.11.23 -t 5

-F 指定需要传输的文件

-T 指定ttl值

3.1.4 TCP吞吐测试

PC端作为server, 板端作为client。Windows中打开cmd命令行, 输入iperf -s以server模式运行iperf, 如下图所示:

```
C:\WINDOWS\system32\cmd.exe - iperf -s
D:\Packages>cd iperf-2.0.9-win64
D:\Packages\iperf-2.0.9-win64>iperf -s
-----
Server listening on TCP port 5001
TCP window size: 208 KByte (default)
-----
[ 4] local 192.168.50.249 port 5001 connected with 192.168.50.243 port 49902
[ ID] Interval      Transfer    Bandwidth
[ 4] 0.0-10.0 sec   112 MBytes  93.5 Mbits/sec
```

Linux板端以client模式运行iperf，测试10秒，如下：

```
/ # ./iperf -c 192.168.50.249 -i 1 -t 10
-----
Client connecting to 192.168.50.249, TCP port 5001
TCP window size: 128 KByte (default)
-----
[ 3] local 192.168.50.243 port 49902 connected with 192.168.50.249
port 5001
[ ID] Interval          Transfer      Bandwidth
[ 3] 0.0- 1.0 sec    11.1 MBytes  93.3 Mbits/sec
[ 3] 1.0- 2.0 sec    11.2 MBytes  94.4 Mbits/sec
[ 3] 2.0- 3.0 sec    11.0 MBytes  92.3 Mbits/sec
[ 3] 3.0- 4.0 sec    11.2 MBytes  94.4 Mbits/sec
[ 3] 4.0- 5.0 sec    11.2 MBytes  94.4 Mbits/sec
```

3.1.5 UDP丢包测试

PC端作为server，板端作为client。Windows中打开cmd命令行，输入iperf **-**s
-u以server模式运行iperf，如下图所示：

```
C:\WINDOWS\system32\cmd.exe - iperf -s -u
D:\Packages>cd iperf-2.0.9-win64
D:\Packages\iperf-2.0.9-win64>iperf -s -u
-----
Server listening on UDP port 5001
Receiving 1470 byte datagrams
UDP buffer size: 208 KByte (default)
-----
[ 3] local 192.168.50.249 port 5001 connected with 192.168.50.243 port 57894
[ 4] local 192.168.50.249 port 5001 connected with 192.168.1.5 port 52562
[ ID] Interval      Transfer    Bandwidth    Jitter    Lost/Total Datagrams
[ 3] 0.0-10.0 sec  1.25 MBytes  1.05 Mbits/sec  0.418 ms    0/ 893 (0%)
```

Linux板端以client模式运行iperf，测试10秒，如下：

```
/ # ./iperf -c 192.168.50.249 -i 1 -t 10 -u
-----
Client connecting to 192.168.50.249, UDP port 5001
Sending 1470 byte datagrams, IPG target: 11215.21 us (kalman
adjust)
UDP buffer size: 512 KByte (default)
-----
[ 3] local 192.168.50.243 port 57894 connected with 192.168.50.249
port 5001
[ ID] Interval          Transfer      Bandwidth
[ 3] 0.0- 1.0 sec      131 KBytes    1.07 Mbits/sec
```

```
[ 3] 1.0- 2.0 sec 128 KBytes 1.05 Mbits/sec
[ 3] 2.0- 3.0 sec 128 KBytes 1.05 Mbits/sec
[ 3] 3.0- 4.0 sec 128 KBytes 1.05 Mbits/sec
[ 3] 4.0- 5.0 sec 128 KBytes 1.05 Mbits/sec
[ 3] 5.0- 6.0 sec 128 KBytes 1.05 Mbits/sec
[ 3] 6.0- 7.0 sec 129 KBytes 1.06 Mbits/sec
[ 3] 7.0- 8.0 sec 128 KBytes 1.05 Mbits/sec
[ 3] 8.0- 9.0 sec 128 KBytes 1.05 Mbits/sec
[ 3] 9.0-10.0 sec 128 KBytes 1.05 Mbits/sec
[ 3] 0.0-10.0 sec 1.25 MBytes 1.05 Mbits/sec
[ 3] Sent 893 datagrams
[ 3] Server Report:
[ 3] 0.0-10.0 sec 1.25 MBytes 1.05 Mbits/sec 0.417 ms 0/
893 (0%)
```

4. 典型FAQ举例

Q：如何添加ephy的网口驱动能力？

[SSD20X][SSD212][SSD22X]适用

```
riu_r 0x31 0x2E
```

```
riu_w 0x31 0x2E 0x7001 //将bit0 set 1
```

```
echo swing_100 2 > /sys/devices/virtual/mstar/emac0/turndrv //增强网口驱动能力
```

```
riu_r 0x33 0x42 //查看当前网络驱动幅值
```

Q：如果有一些超长网线(>200m)的场景怎么办？

[SSD20X][SSD212][SSD22X]适用

传输距离超过100m属于超规格，无法保证可靠性，但是可以尝试将速度限定到10M。

```
riu_w 0x31 0x04 0x61
```

```
riu_w 0x31 0x00 0x1200
```

```
echo max > /sys/class/mstar/emac0/turndrv
```